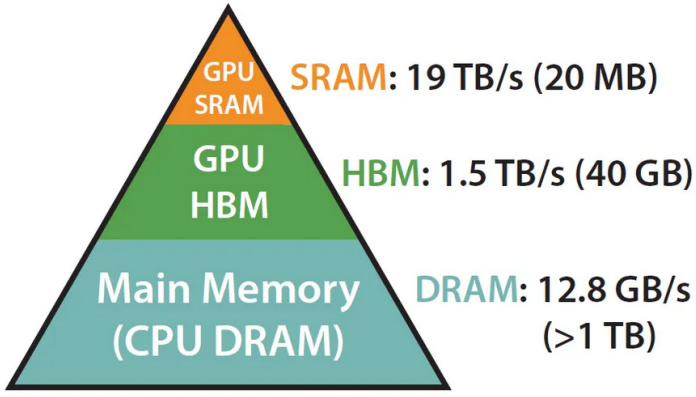
Deep Learning for NLP

Training LLMs
How to reduce compute and memory



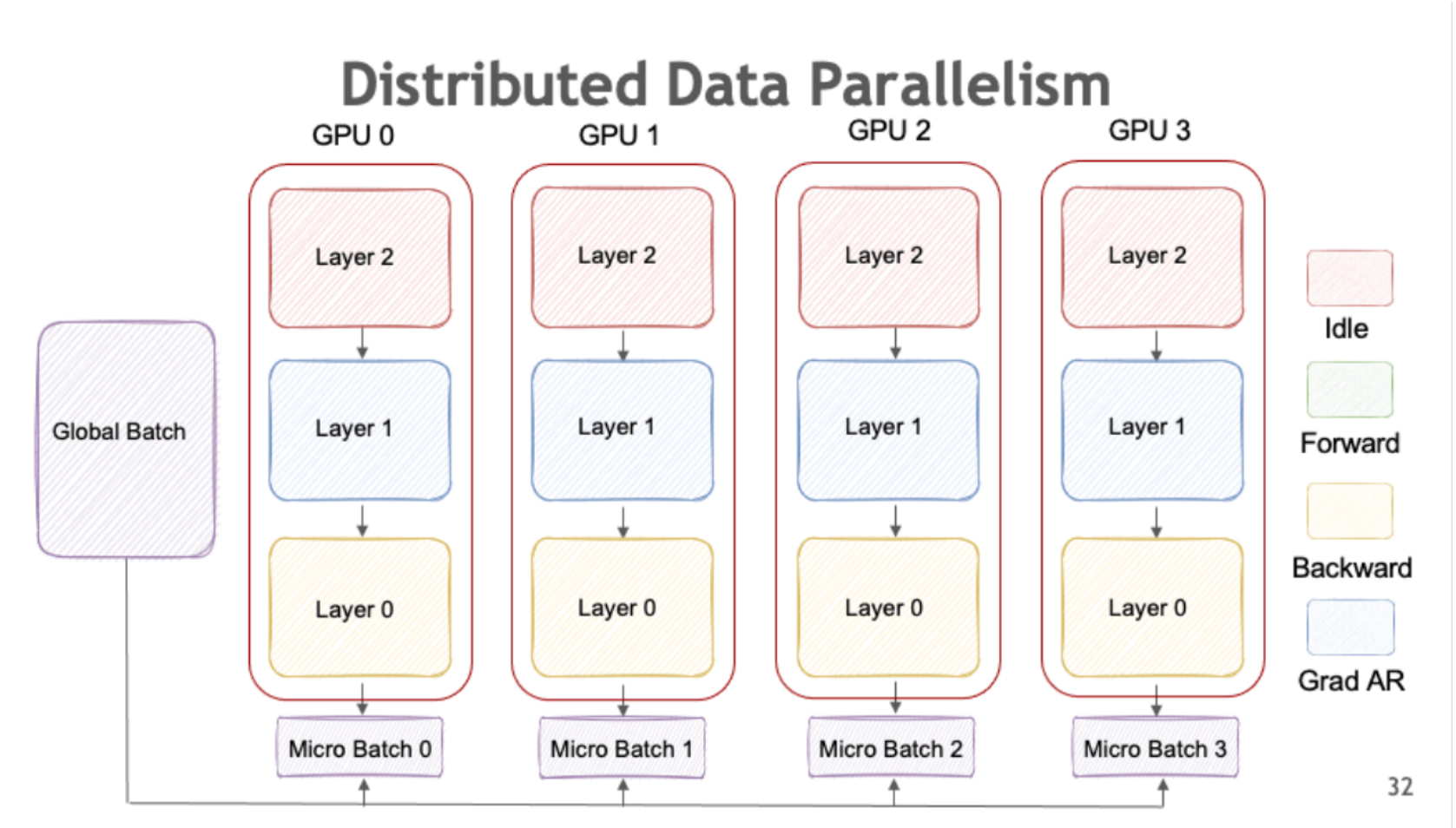
Learning goals

- Learn about different techniques to reduce compute and memory
- Learn about distributed training with data/tensor parallelism
- Learn about FlashAttention

DISTRIBUTED TRAINING

- Training LLMs on several GPUs is faster than on one, but we need to figure out how to distribute.
- Avoiding OOM (out of memory) issues
- **Data parallelism:** split the data on different model replicas
- **Tensor parallellism:** split model parameters accross GPUs

DATA PARALLELISM (1) (ANIMATED GIF!)

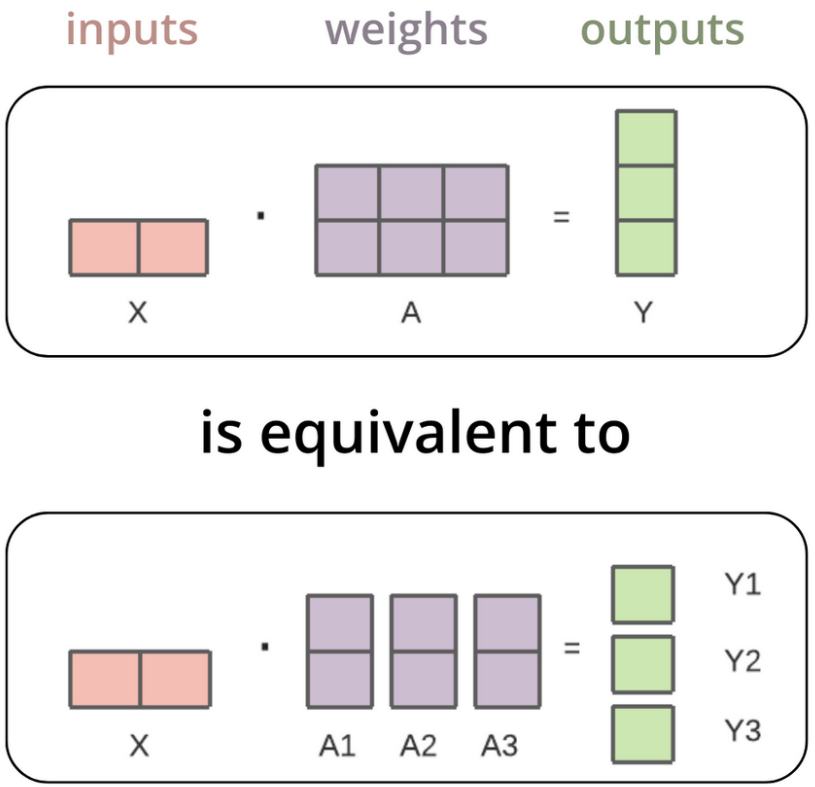


Source: Nvidia

DATA PARALLELISM (2)

- **Data Splitting**
  - Dataset divided into smaller chunks. Each chunk assigned to a different GPU.
  - Each node processes a different subset of the data in parallel.
- **Model Replication**
  - Each GPU has a replica of the neural network model
  - These replicas are trained independently on their respective data subsets
- **Gradient Aggregation (All Reduce)**
  - Gradients are computed locally on each node and then averaged across nodes.
- **Parameter Synchronization**
  - Model parameters are updated synchronously across nodes.
  - This ensures that all model replicas remain consistent with each other after each update step.

TENSOR PARALLELISM (1)



Source: Nvidia

TENSOR PARALLELISM (2)

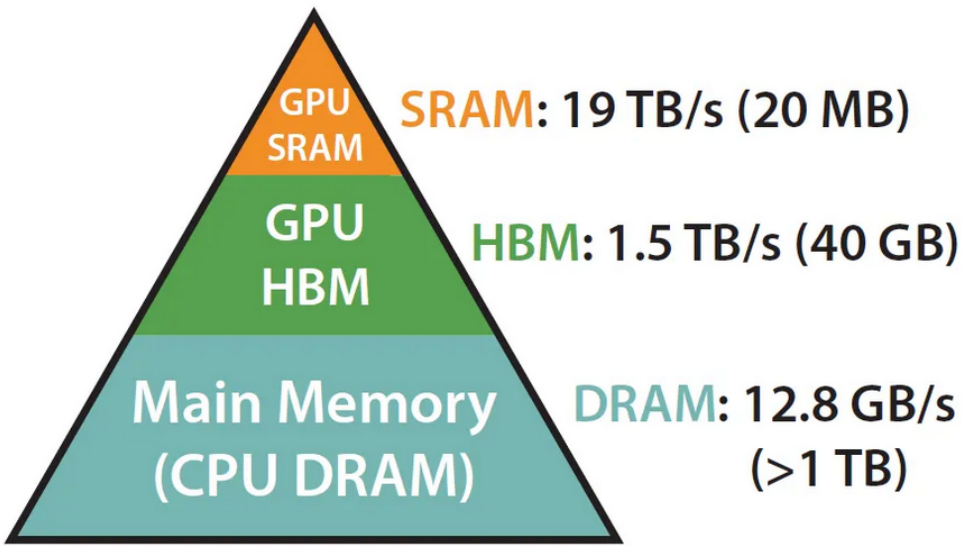
- **Model Partitioning:**
  - The model's layers or tensors are split across multiple devices
  - Different parts of the model are assigned to different devices, enabling them to work on separate portions of the computations simultaneously
- **Forward and Backward Passes:**
  - During the forward pass, each device processes its portion of the tensors with intermediate results passed between devices
  - In the backward pass, gradients are computed in the reverse order, again with necessary data transfers between devices
- **Parameter Updates:**
  - Parameter updates can be performed independently on each device for the parameters they own
  - After each update step, the devices synchronize to ensure consistency across the distributed model

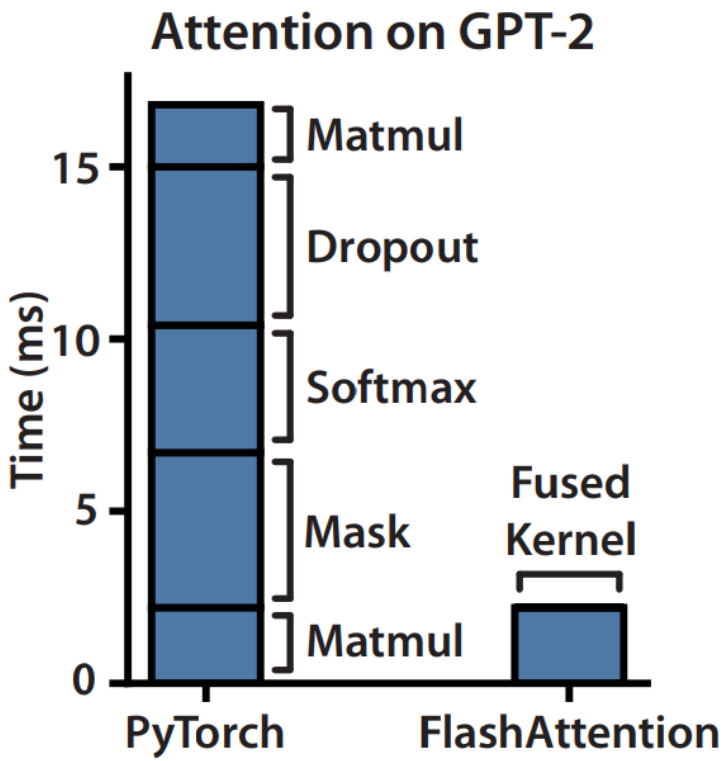
FlashAttention

Fast and Memory-Efficient Exact Attention with IO-Awareness

- Memory-efficient
  - Reducing from $O(n^2)$ to $O(n)$
- Therefore: IO aware
  - Reducing memory load/store operations
- Therefore: much faster than regular implementation
  - The bigger the attention matrices, the more you gain (until $O(n)$ no longer fits).
- Exact
  - Same as “vanilla attention”, not an approximation

GPU MEMORY HIERARCHY





▸ Source: Dao, 2022

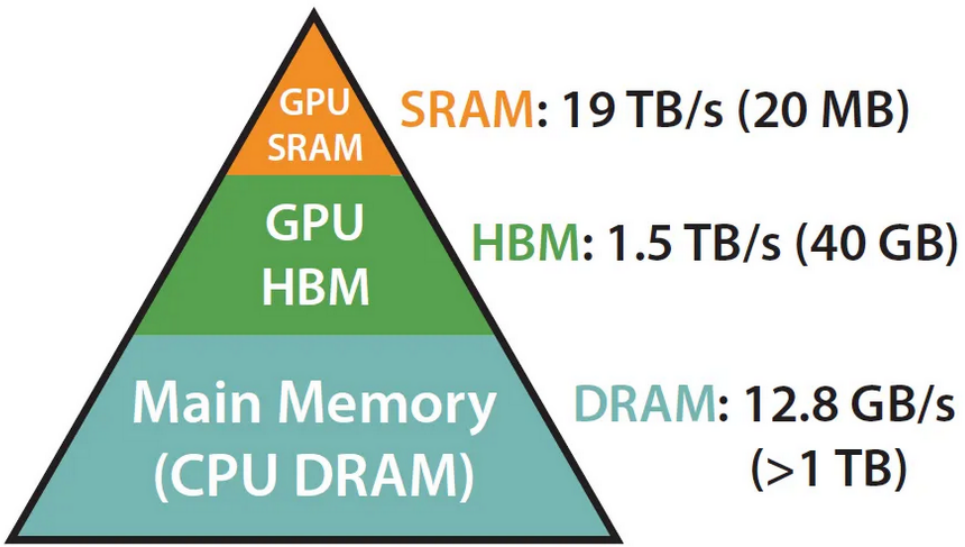
COMPUTING CONSIDERATIONS

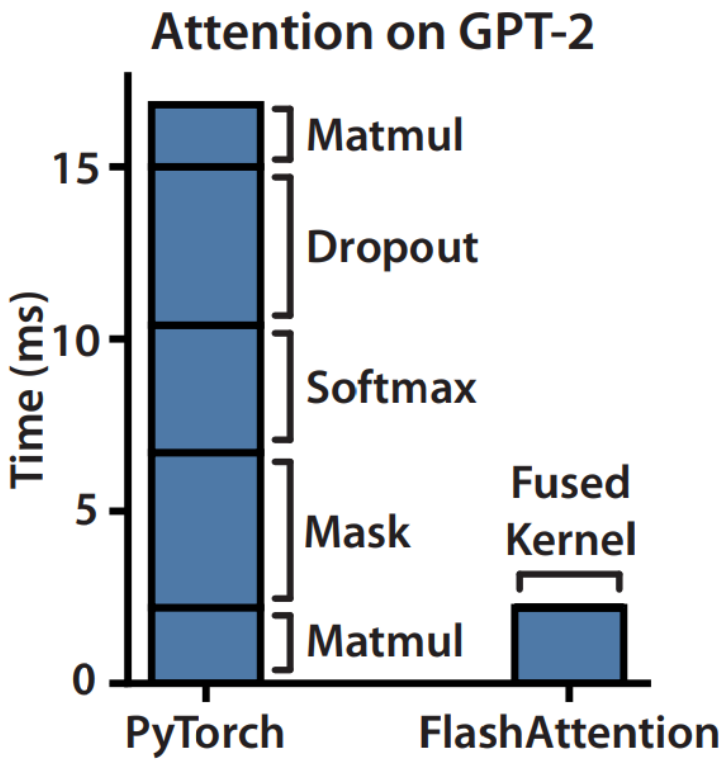
- GPU compute has been growing faster than memory bandwidth
  - GPU has to wait for data
- Transformer operations are memory-bound
  - Elementwise operations with high memory access
- IO aware means reducing memory load/store operations

FLASH ATTENTION

- Tiling or chunking of matrices into blocks
- Operation fusion
  - For example: first matrix multiplication (query-key), softmax, second matrix multiplication (weight-value)
- This minimizes redundant swapping in and out of data
- General idea: you avoid that the large intermediate tensors (e.g., pre-softmax) are written to HBM and then read back into SRAM memory

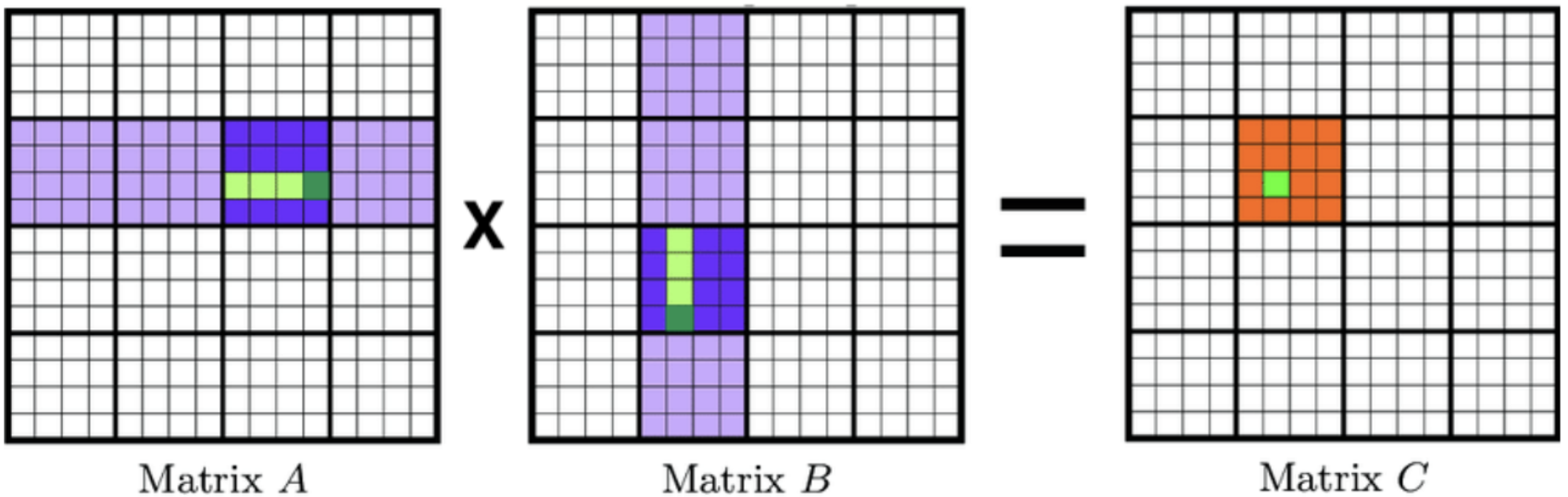
SRAM VS HBM / FUSED KERNEL



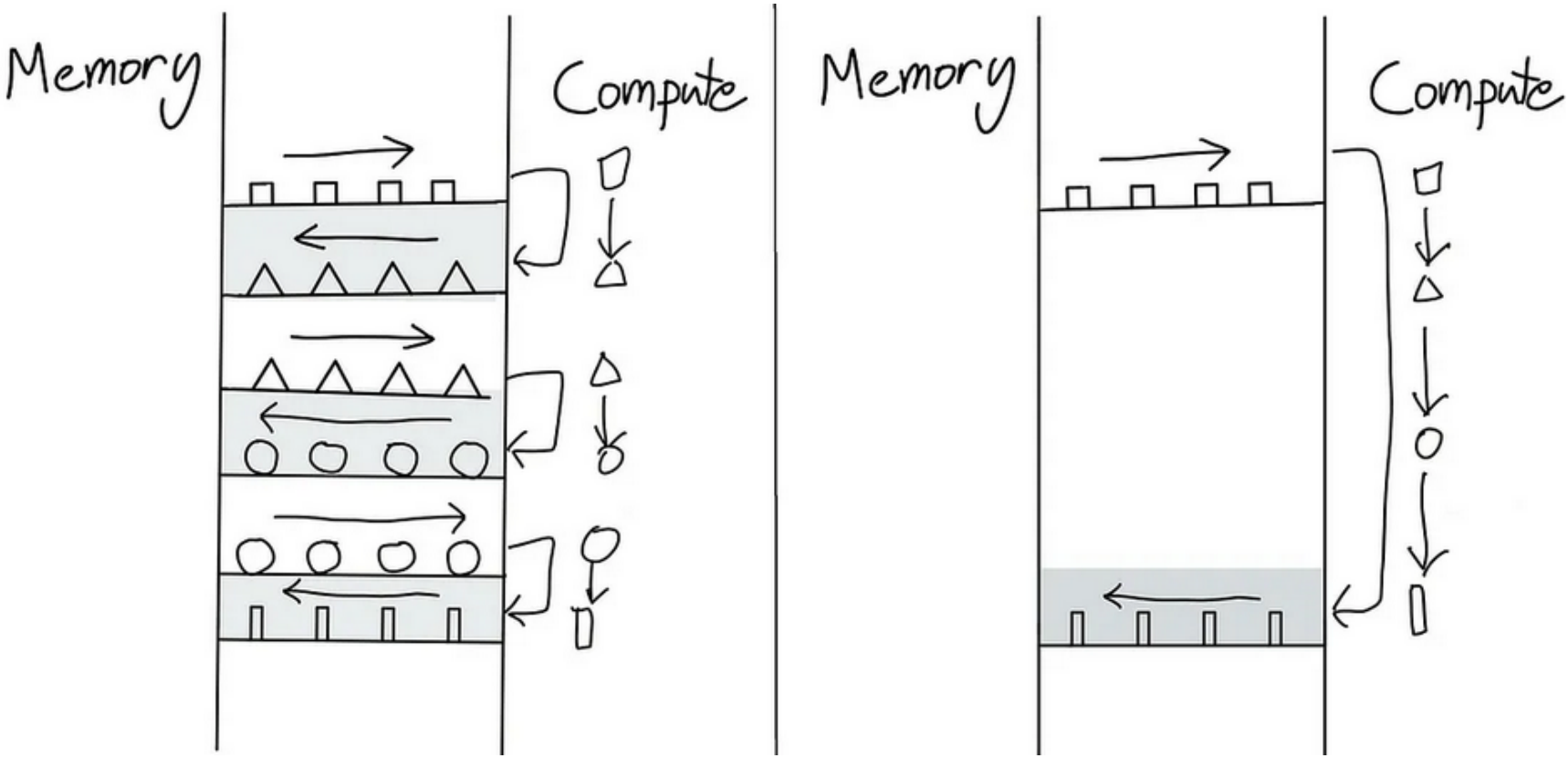


▸ Source: Dao, 2022

TILING



OPERATION FUSION



Source: `https://horace.io/brrr_intro.html`

LIMITATIONS AND PROSPECTS

- FlashAttention requires implementing “custom” attention in CUDA
  - A new CUDA kernel for each new attention implementation
  - CUDA is lower-level than PyTorch
  - Implementation not transferable accross GPUs
- Towards IO-Aware Deep Learning
  - Extending beyond attention
  - DeepSeek